API de clientes, compras y abonos para Kenxon Car Sound

Sara Mariana Garzon y Gabriela Ocampo Morales

Facultad de Ingenierías, Universidad Católica de Oriente

Programación Orientada a Objetos – Ingeniería de Sistemas

Juan Ricardo Cogollo Oyola

5 de noviembre de 2026

API de clientes, compras y abonos para Kenxon Car Sound

Parte 1. Contexto general

Introducción

Este documento presenta el análisis, diseño e implementación de una API en C# para Kenxon Car Sound, empresa de Rionegro (Antioquia) dedicada a la venta de sistemas de audio y accesorios de seguridad para vehículos. La API permite al jefe y a la secretaria registrar clientes mayoristas, sus compras y los abonos diarios con los que cada cliente completa el pago, aplicando la regla del abono inicial mínimo. El proyecto aplica los conceptos del curso de Programación Orientada a Objetos: clases, encapsulamiento, herencia, polimorfismo e interfaces. Además, incorpora autenticación mediante JWT con tres roles (Jefe, Secretaria y Empleado) y almacena la información en archivos JSON, sin utilizar una base de datos.

Contexto organizacional

Kenxon Car Sound es una marca reconocida en el mercado colombiano del car audio y de los sistemas de sonido y seguridad automotriz, con sede en Rionegro, Antioquia. Ofrece sistemas profesionales de audio, componentes de sonido de alta fidelidad (amplificadores, medios y subwoofers) y accesorios de seguridad vehicular. Sus clientes son mayoristas y pagan sus compras mediante abonos hasta completar el valor total; el abono inicial mínimo es de $8.000.000 COP. Los datos básicos de la empresa son los siguientes:

- Ubicación: Rionegro, Antioquia, Colombia.
- Personal: el jefe y la secretaria, encargada de recibir los abonos de las compras diarias. Existen además otros cargos: asesores de venta, instaladores y bodega.

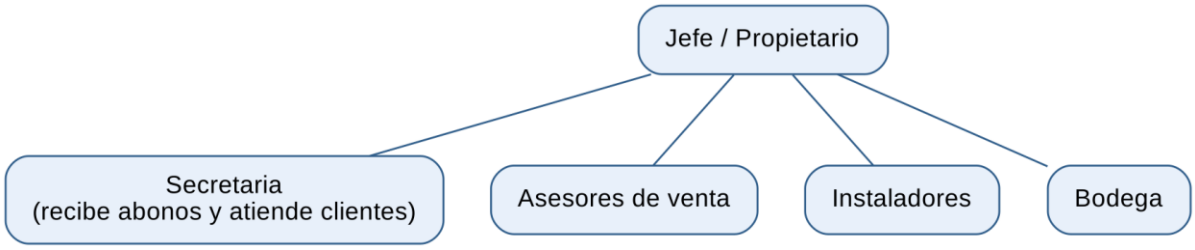
Organigrama

La estructura actual de la empresa se presenta en la Figura 1 y comprende los siguientes cargos:

- Jefe / Propietario.
- Secretaria (recibe abonos y atiende clientes).
- Otros cargos: asesores de venta, instaladores y bodega.

Figura 1

Organigrama de Kenxon Car Sound



Situación problemática

Hoy el control de compras y abonos se lleva de forma manual en un libro físico. Esto genera los siguientes problemas:

- Los datos de los clientes (nombre, cédula, celular y dirección) están dispersos o duplicados.
- Los abonos diarios se anotan a mano, por lo que es fácil equivocarse en el saldo pendiente.
- La regla del abono inicial mínimo de $8.000.000 COP se verifica manualmente y puede omitirse.
- El jefe no puede consultar rápidamente la cartera pendiente ni el estado de cuenta de cada cliente.
- No existe un historial verificable de pagos que respalde al cliente y a la empresa.

Justificación de la solución

Una API reduce errores humanos, valida automáticamente el abono inicial mínimo y entrega al jefe información confiable de la cartera. Además, así puede conectarse luego a una

aplicación de escritorio, web o móvil. Desde lo académico, el caso permite modelar objetos con estado y comportamiento (clientes, compras, abonos), aplicar herencia e interfaces y construir una solución con buenas prácticas, pruebas unitarias, autenticación JWT y almacenamiento sencillo en archivos JSON.

Parte 2. Solución propuesta

Descripción de la propuesta de solución

Se construirá una API en ASP.NET Core (C#), desarrollada en Visual Studio a partir de la plantilla ASP.NET Core Web API y organizada mediante Controllers, Models, Interfaces y Repositorios, con autenticación JWT. Sus módulos son:

- **Seguridad:** inicio de sesión con usuario y contraseña mediante JWT y control de acceso por roles (Jefe, Secretaria y Empleado).
- **Clientes:** registro, edición y consulta de la información de los clientes mayoristas.
- **Compras:** registro de compras junto con su abono inicial, que debe ser de mínimo $8.000.000 COP; calcula automáticamente el saldo pendiente.
- **Abonos:** registro de pagos posteriores, actualización del saldo y cambio automático del estado de la compra.
- **Reportes:** consulta de estados de cuenta, comprobantes de abono y cartera pendiente.

La información se almacenará en archivos JSON (usuarios.json, clientes.json, compras.json y abonos.json). Cada repositorio carga su archivo en una lista en memoria al iniciar la API y lo vuelve a guardar cada vez que se registra o modifica un dato. El acceso a los datos se realizará mediante Interfaces y Repositorios, lo que permite aplicar los principios de la Programación Orientada a Objetos.

Listado de actores identificados del sistema

En la Tabla 1 se describen los actores que interactúan con el sistema.

Tabla 1

Actores del sistema

Actor	Descripción
Jefe (administrador)	Usuario principal. Registra y edita clientes, registra compras y abonos, y consulta clientes, estados de cuenta, comprobantes y cartera.
Secretaria	Usuario operativo. Recibe y registra los abonos diarios, registra y edita clientes, registra compras y consulta estados de cuenta, comprobantes y cartera.
Empleado	Usuario de solo consulta. Puede consultar clientes, estados de cuenta, comprobantes de abono y cartera, pero no registra ni modifica información.
Cliente (mayorista)	Actor externo. No usa la API; compra, abona y únicamente recibe el comprobante de abono.

Listado de funcionalidades

La Tabla 2 enumera las funcionalidades previstas para la API.

Tabla 2

Funcionalidades del sistema

Código	Funcionalidad
F01	Iniciar sesión (obtener token JWT)
F02	Registrar cliente (nombre, cédula, celular, dirección)
F03	Editar cliente
F04	Consultar cliente
F05	Registrar compra con abono inicial (mínimo $8.000.000 COP)
F06	Registrar abono a una compra
F07	Consultar saldo y estado de cuenta del cliente
F08	Generar comprobante de abono
F09	Consultar cartera pendiente

Relación entre actores y funcionalidades

La Tabla 3 muestra qué funcionalidades puede utilizar cada actor. El Cliente no opera la API; solo recibe el comprobante de abono.

Tabla 3

Relación entre actores y funcionalidades

Funcionalidad	Jefe	Secretaria	Empleado	Cliente
F01 Iniciar sesión	Sí	Sí	Sí	No
F02 Registrar cliente	Sí	Sí	No	No
F03 Editar cliente	Sí	Sí	No	No
F04 Consultar cliente	Sí	Sí	Sí	No
F05 Registrar compra	Sí	Sí	No	No
F06 Registrar abono	Sí	Sí	No	No
F07 Estado de cuenta	Sí	Sí	Sí	No
F08 Comprobante de abono	Sí	Sí	Sí	Lo recibe
F09 Cartera pendiente	Sí	Sí	Sí	No

Listado de objetos identificados para el sistema

Las clases identificadas, con sus atributos y responsabilidades, se resumen en la Tabla 4.

Tabla 4

Clases del sistema

Clase	Atributos principales	Responsabilidades
Persona (abstracta)	Id, Cedula, Nombre, Celular, Direccion	Base común de Cliente y Usuario; define el método abstracto ObtenerDescripcion
Usuario	NombreUsuario, ContrasenaHash, Rol	Autenticarse y validar permisos según su rol
Cliente	(hereda de Persona)	Representar al cliente mayorista que realiza compras
Compra	Id, Fecha, ClienteId, Descripcion, Total, Abonos, EstadoPago	Calcular saldo, agregar abonos y cambiar el estado de pago
Abono	Id, Fecha, Monto, CompraId, UsuarioId, SaldoPosterior	Registrar cada pago realizado
Rol (enumeración)	Jefe, Secretaria, Empleado	Definir los permisos
EstadoPago (enumeración)	Parcial, Pagada	Indicar el estado de la compra
IUsuarioRepositorio	Métodos de consulta	Contrato para la gestión de usuarios
IClienteRepositorio	Métodos de consulta, registro y edición	Contrato para la gestión de clientes
ICompraRepositorio	Métodos de consulta, registro y edición	Contrato para la gestión de compras
IAbonoRepositorio	Métodos de consulta y registro	Contrato para la gestión de abonos
UsuarioRepositorio	rutaArchivo, lista	Implementa las operaciones de usuarios sobre usuarios.json
ClienteRepositorio	rutaArchivo, lista	Implementa las operaciones de clientes sobre clientes.json
CompraRepositorio	rutaArchivo, lista	Implementa las operaciones de compras sobre compras.json
AbonoRepositorio	rutaArchivo, lista	Implementa las operaciones de abonos sobre abonos.json

Las relaciones principales son las siguientes: un Cliente realiza varias Compras; una Compra contiene uno o más Abonos (composición), y el primero de ellos es el abono inicial que se registra junto con la compra; cada Abono lo registra un Usuario. Como regla de negocio, el abono inicial de cada compra debe ser de al menos $8.000.000 COP y no puede superar el total

de la compra, pues todos los clientes son mayoristas. Los abonos posteriores deben ser mayores a cero y no pueden superar el saldo pendiente. Como la compra siempre nace con un abono inicial, su estado solo puede ser Parcial o Pagada. El polimorfismo se aplica en el método ObtenerDescripcion, que Cliente y Usuario implementan de forma distinta a partir de la clase abstracta Persona.

Diagrama de casos de uso

La Figura 2 presenta el diagrama de casos de uso. El Jefe y la Secretaria se asocian con los casos de uso F01 a F09. El Empleado se asocia únicamente con F01, F04, F07, F08 y F09, que son de consulta. El Cliente no usa la API y solo se asocia con F08, pues recibe el comprobante de abono. *Registrar compra* incluye *Validar abono inicial mínimo*, y *Registrar abono* incluye *Actualizar saldo*.

Figura 2

Diagrama de casos de uso

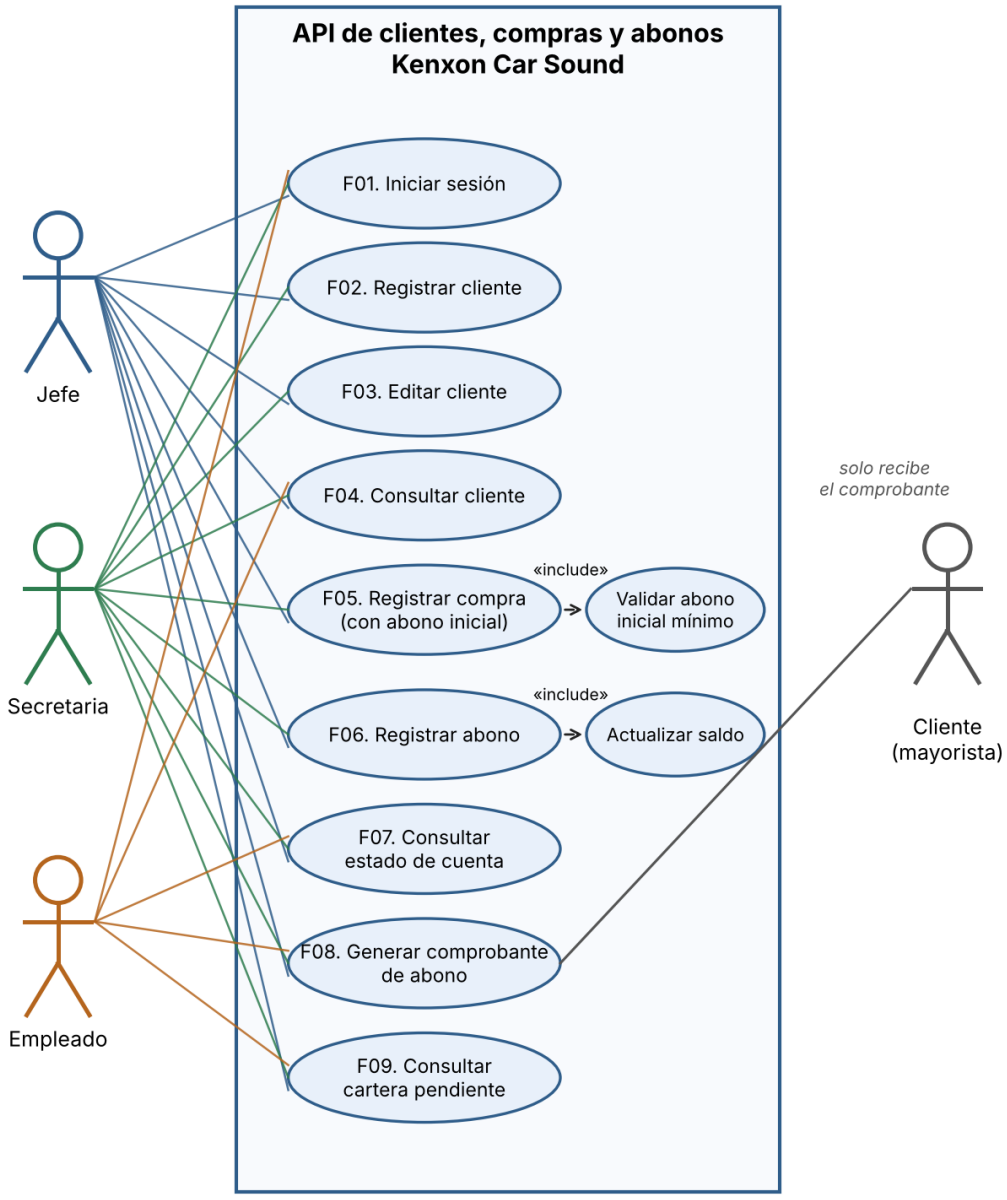


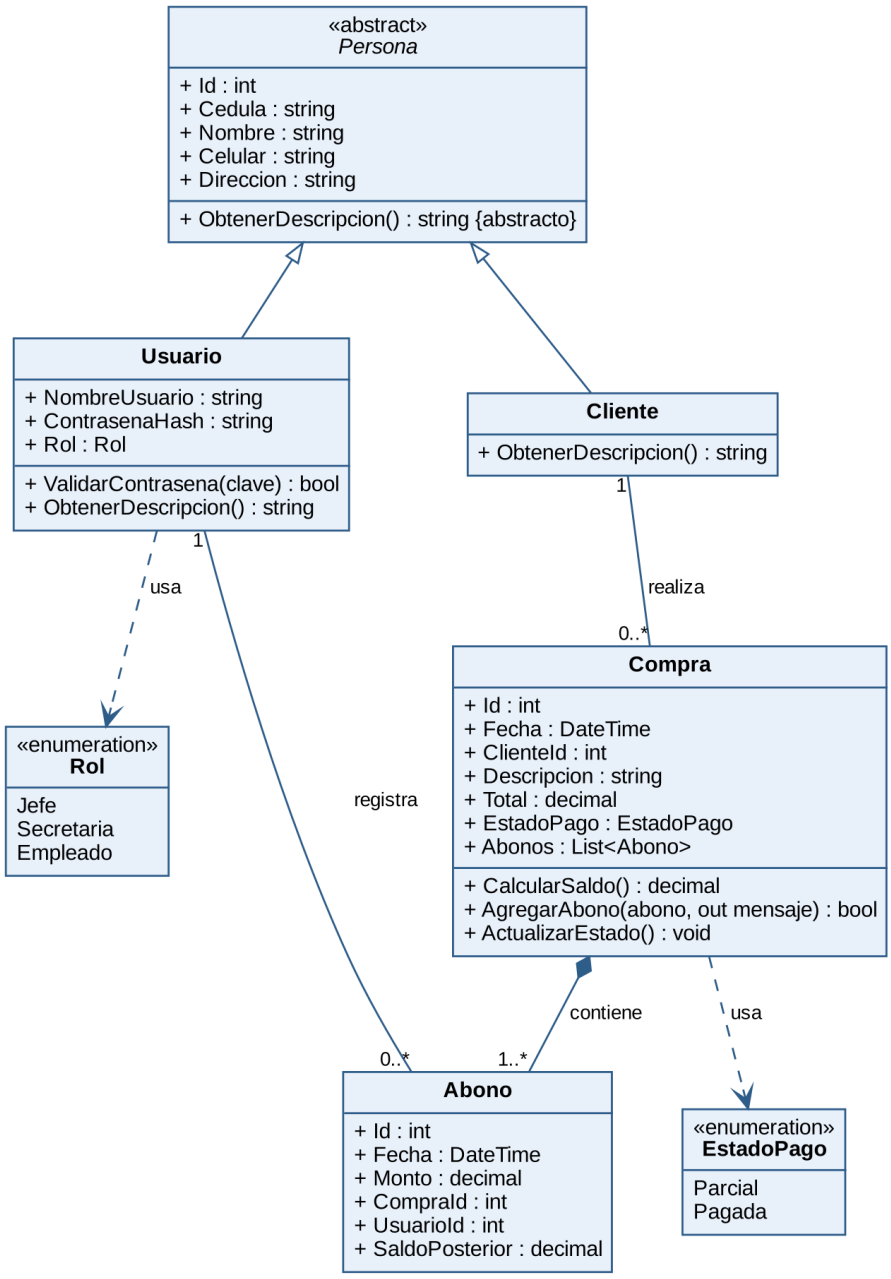
Diagrama de clases

En la Figura 3 se muestra el modelo de dominio. La clase abstracta Persona (Id, Cedula, Nombre, Celular y Direccion) actúa como clase base para Usuario y Cliente. Las entidades

Cliente, Compra y Abono representan la información principal del negocio, y las enumeraciones Rol y EstadoPago limitan los valores posibles de esos atributos.

Figura 3

Diagrama de clases del modelo de dominio



La Figura 4 muestra las interfaces y los repositorios. Cada repositorio implementa su interfaz, mantiene una lista en memoria y se encarga de leer y guardar su archivo JSON. Esta

estructura mantiene un bajo acoplamiento: los controladores dependen de las interfaces y no de la forma en que se guardan los datos.

Figura 4

Diagrama de clases de interfaces y repositorios

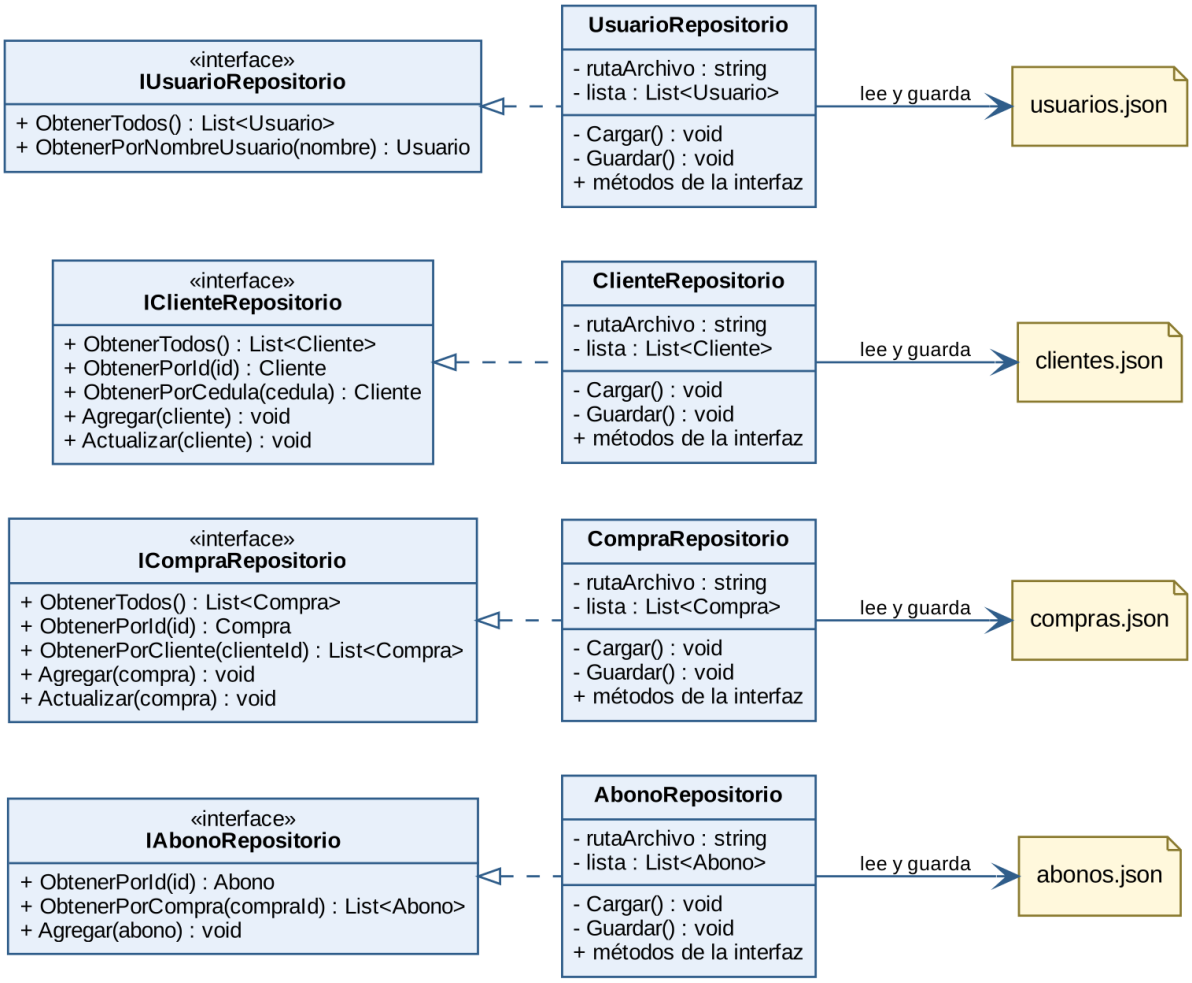


Diagrama de secuencia

La Figura 5 ilustra el proceso de registro de un abono posterior al inicial:

1. La Secretaria inicia sesión y recibe un token JWT.
2. Envía la solicitud para registrar un abono con el identificador de la compra y el monto.
3. AbonosController valida el token y el rol.

4. El controlador consulta la compra mediante ICompraRepositorio y los abonos existentes mediante IAbonoRepositorio.
5. El controlador le pide a la compra agregar el abono (AgregarAbono).
6. La compra valida que el monto sea mayor a cero, que no supere el saldo pendiente y que no esté ya pagada.
7. Si es válido, actualiza el saldo y el estado: si el saldo llega a cero pasa a **Pagada**; de lo contrario, permanece **Parcial**.
8. El controlador guarda el abono y la compra mediante los repositorios, que actualizan los archivos JSON.
9. El sistema devuelve la confirmación con el saldo actualizado, o un mensaje de error si alguna validación falló.

Figura 5

Diagrama de secuencia del registro de un abono

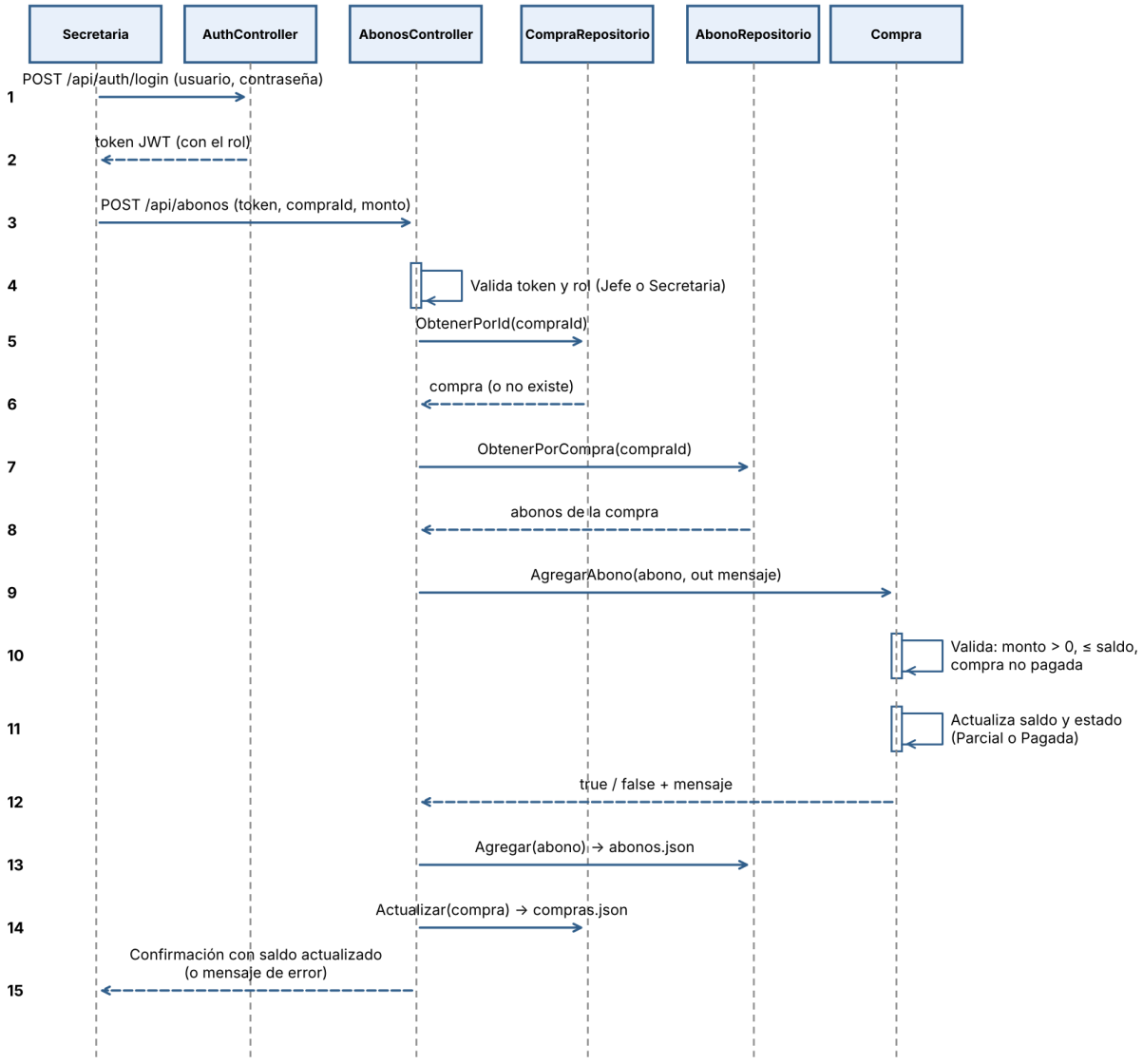


Diagrama de actividades

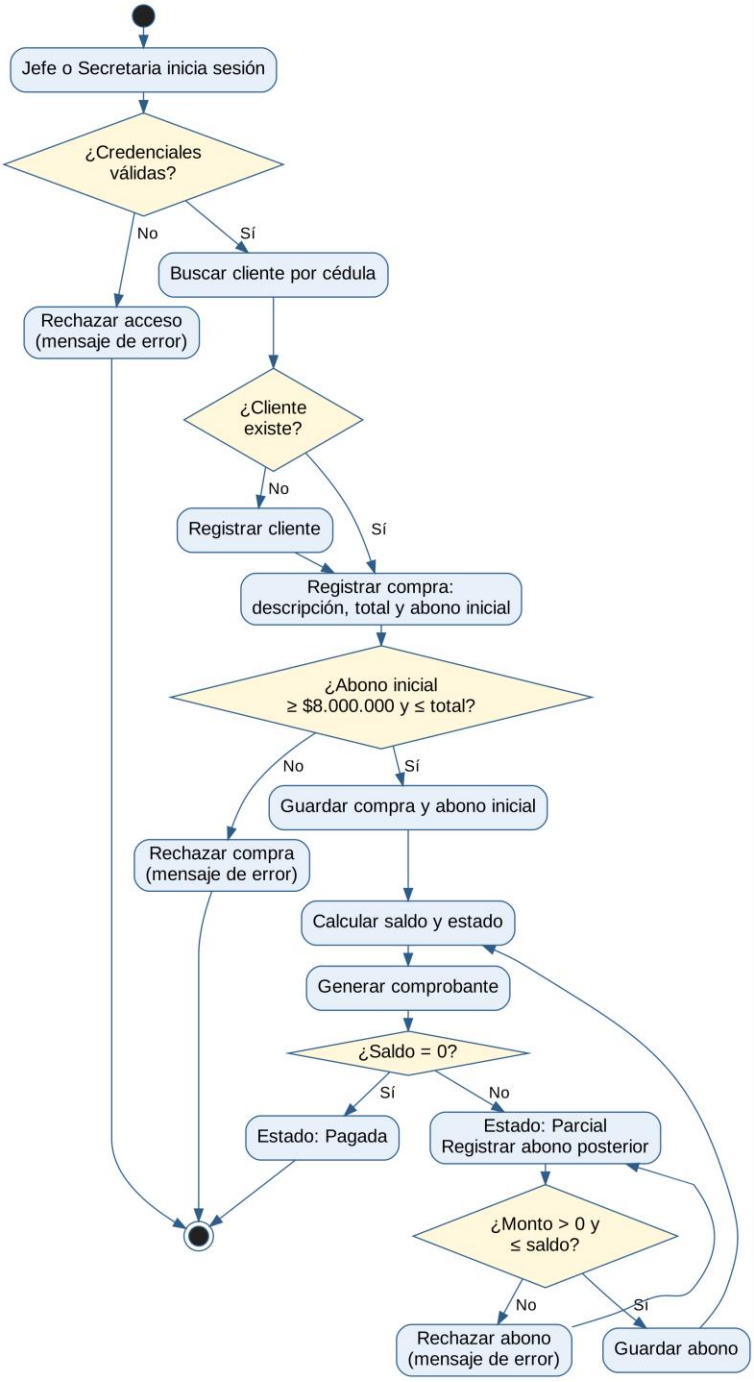
La Figura 6 representa el proceso de compra con abonos, que sigue estos pasos:

1. El Jefe o la Secretaria inicia sesión; si las credenciales no son válidas, se rechaza el acceso.
2. Busca al cliente por cédula y, si no existe, lo registra.

3. Registra la compra indicando su descripción, su valor total y el abono inicial.
4. El sistema valida que el abono inicial sea de mínimo $8.000.000 COP y no supere el total; si no cumple, rechaza la compra.
5. Si la validación es correcta, guarda la compra y el abono inicial, y calcula el saldo y el estado.
6. Se genera el comprobante correspondiente.
7. Si el saldo es cero, la compra queda en estado **Pagada** y el proceso termina; de lo contrario, queda en estado **Parcial**.
8. Cuando el cliente paga de nuevo, se registra un abono posterior; el sistema valida que el monto sea mayor a cero y no supere el saldo.
9. Si es válido, se guarda el abono, se recalcula el saldo y el estado, y se genera un nuevo comprobante.

Figura 6

Diagrama de actividades del proceso de compra con abonos



Parte 3. Implementación en C#

Arquitectura

Se usará una API REST en ASP.NET Core con una arquitectura sencilla organizada en las siguientes carpetas:

- **Controllers:** AuthController, ClientesController, ComprasController, AbonosController y ReportesController.
- **Models:** entidades del dominio (Persona, Usuario, Cliente, Compra, Abono) y las enumeraciones Rol y EstadoPago. Aquí se encuentran las validaciones de negocio.
- **Interfaces:** IUsuarioRepositorio, IClienteRepositorio, ICompraRepositorio e IAbonoRepositorio.
- **Repositorios:** implementan las interfaces y leen y guardan la información en archivos JSON.
- **Data:** carpeta que contiene los archivos usuarios.json, clientes.json, compras.json y abonos.json.

Los endpoints principales se presentan en la Tabla 5.

Tabla 5

Endpoints principales de la API

Método	Ruta	Acceso	Función
POST	/api/auth/login	Público	Iniciar sesión y obtener JWT
POST	/api/clientes	Jefe, Secretaria	Registrar cliente
GET	/api/clientes	Jefe, Secretaria, Empleado	Consultar clientes
GET	/api/clientes/{id}	Jefe, Secretaria, Empleado	Consultar un cliente
PUT	/api/clientes/{id}	Jefe, Secretaria	Editar cliente
POST	/api/compras	Jefe, Secretaria	Registrar compra con abono inicial
POST	/api/abonos	Jefe, Secretaria	Registrar abono
GET	/api/abonos/{id}/comprobante	Jefe, Secretaria, Empleado	Generar comprobante de abono
GET	/api/clientes/{id}/estado-cuenta	Jefe, Secretaria, Empleado	Consultar saldo y abonos
GET	/api/reportes/cartera	Jefe, Secretaria, Empleado	Consultar cartera pendiente

Pruebas unitarias

Se utilizará **xUnit** para la implementación de las pruebas unitarias. Las pruebas mínimas previstas son:

- Validar el registro correcto de un cliente y rechazar una cédula repetida.
- Calcular correctamente el saldo al registrar una compra con su abono inicial.
- Rechazar un abono inicial menor a $8.000.000 COP.
- Rechazar un abono inicial mayor al total de la compra.
- Rechazar abonos con monto cero, negativo o superior al saldo pendiente.
- Actualizar correctamente el saldo después de registrar un abono.
- Cambiar el estado a **Parcial** cuando exista saldo pendiente.
- Cambiar el estado a **Pagada** cuando el saldo llegue a cero.

- Validar el inicio de sesión con credenciales correctas e incorrectas.
- Verificar los permisos de los roles Jefe, Secretaria y Empleado.
- Verificar que los repositorios guardan y recuperan los datos desde el archivo JSON.

Autenticación y autorización

La API utilizará JWT (JSON Web Token) bajo el esquema Bearer. Los usuarios accederán al sistema mediante nombre de usuario y contraseña. Una vez autenticados, recibirán un token que incluye su rol y que deberán enviar para acceder a los recursos protegidos. Los tres usuarios del sistema (Jefe, Secretaria y Empleado) estarán precargados en el archivo usuarios.json; la API no ofrece opciones para crear usuarios. Los roles definidos son:

- **Jefe:** puede realizar todas las funcionalidades.
- **Secretaria:** puede registrar clientes, compras y abonos, editar clientes y consultar la información.
- **Empleado:** tiene permisos únicamente de consulta.

Las contraseñas se almacenarán de forma segura mediante hash y los permisos se gestionarán utilizando atributos de autorización sobre los endpoints del sistema.

Almacenamiento de datos

El sistema no utiliza base de datos. La información se guarda de forma permanente en archivos JSON dentro de la carpeta Data, como se resume en la Tabla 6.

Tabla 6

Archivos de datos del sistema

Archivo	Contenido	Repositorio responsable
usuarios.json	Los tres usuarios del sistema con su rol y contraseña en hash	UsuarioRepositorio
clientes.json	Datos de los clientes mayoristas	ClienteRepositorio
compras.json	Compras con su total, saldo y estado de pago	CompraRepositorio
abonos.json	Abonos de cada compra, con la fecha, el monto y el usuario que lo registró	AbonoRepositorio

Conclusiones

- Este proyecto nos permitió poner en práctica los conceptos vistos durante el curso de Programación Orientada a Objetos, especialmente el uso de clases, herencia, encapsulamiento, polimorfismo e interfaces, aplicándolos en una situación cercana a un caso real.
- La solución propuesta facilita el control de clientes, compras y abonos, ya que permite organizar mejor la información y reducir los errores que pueden presentarse cuando los registros se realizan de forma manual.
- La autenticación mediante JWT y la definición de los roles Jefe, Secretaria y Empleado permiten controlar el acceso a la información y asignar permisos según las funciones de cada usuario.
- El uso de Interfaces y Repositorios con archivos JSON mantiene una estructura organizada y sencilla, y deja abierta la posibilidad de cambiar el almacenamiento sin modificar el resto del sistema.

- Como mejora futura, el sistema podría ampliarse con una base de datos, reportes más detallados, notificaciones de pago y una interfaz gráfica que facilite aún más su utilización por parte de la empresa.

Referencias

Cogollo, J. R. (2026). *Programación orientada a objetos* [Apuntes de clase].

Anthropic. (2026). *Claude* (Versión actual) [Modelo de lenguaje grande]. https://claude.ai